DAT107 Databaser – våren 2026

Obligatorisk øvelse 4: NoSQL

Lasse Jenssen, 18.04.2026

Rammer

Oppgaven løses i grupper på inntil 4 deltagere. Du kan også levere alene.

NB! Oppgaven skal leveres via en av gruppene i gruppesettet 'Oblig 4 Gr'. Dette gjelder også om du leverer alene (Finn et ledig gruppenummer og legg dere inn selv).

Innleveringer utenfor dette gruppesettet risikerer å ikke bli rettet!!

Vurdering og godkjenning

Skriftlig innlevering i Canvas innen **onsdag 5. mai kl.23:59.**

Leveranse i Canvas

Leveranse i Canvas skal være:

1) **Et PDF-dokument** med navn «dat107-oblig4.pdf»:
   a) NB! I starten av PDF-dokumentet skal det stå
      - gruppenummer
      - navn og studentnummer til alle som er med på innleveringen
   b) Hver besvarelse/deloppgave skal ha overskrift med «Oppgave <nr> <bokstav>»
   c) Skjermdump(er) av løst oppgave (der oppgaven spesifiserer dette)
2) **En Zip-fil** som inneholder genererte filer (f.eks. XML, XML Skjema, XSLT og/eller JSON).

Databaser

Database	Beskrivelse
PostgreSQL	Databasen som benyttes for å løse PostgreSQL relaterte oppgaver kjøres i Docker. Se “Forberedelser: Docker” for nærmere beskrivelse.
MongoDB	Databasen som benyttes for å løse MongoDB relaterte oppgaver kjøres i Docker. Se “Forberedelser: Docker” for nærmere beskrivelse.

Bakgrunn

I denne oppgaven tar vi utgangspunkt i spesifikasjonen for et medlemsregister gitt i obligatorisk oppgave 2:

> *«En forening har behov for et system for å administrere medlemmer. Et medlem har medlemsnummer, fornavn, etternavn, et telefonnummer, en epost-adresse, postnummer/sted, gatenavn/husnummer, samt om personen fortsatt er medlem (ikke utmeldt).*
>
> *Foreningen har også en rekke lokallag. Alle medlemmer er med i nøyaktig et lokallag. Lokallag har lag-navn, person som er leder (som også er medlem), postnummer/sted, gatenavn/husnummer for møtelokale.*
>
> *For hvert medlem skal det lagres hvorvidt medlemsavgift har blitt betalt for hvert år. Historiske data skal lagres, dvs., det skal være mulig å sjekke at avgift har blitt betalt for f.eks. 2019, 2020, 2021, osv.»*

Vi har laget følgende datamodell for løsningen som er tenkt kjørt i en PostgreSQL database.

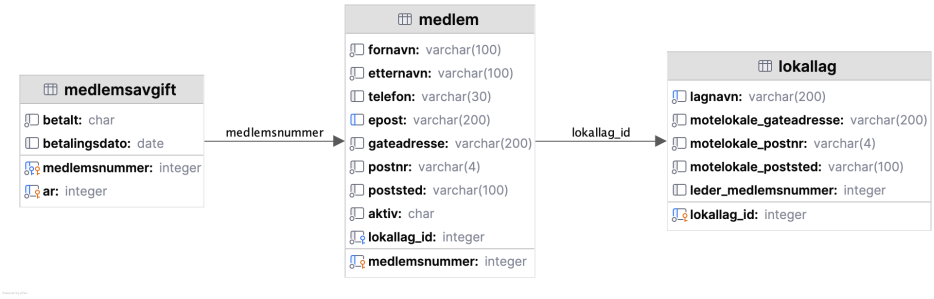


Figure 1 - Datamodell for medlemsregister lokallag

(Oppgaven fortsetter på neste side.)

Forberedelser: Docker

I GitHub repository for «NoSQL» delen (se link under) finner dere mappen «obligatorisk» og filen «obligatorisk/docker-compose.yaml». Docker Compose filen har oppsett for å starte både en PostgreSQL database og en MongoDB database med eksempel data.

Link til GitHub og docker-compose.yaml:

https://github.com/lasjen/hvl-dat107-databaser-V2026/tree/main/obligatorisk

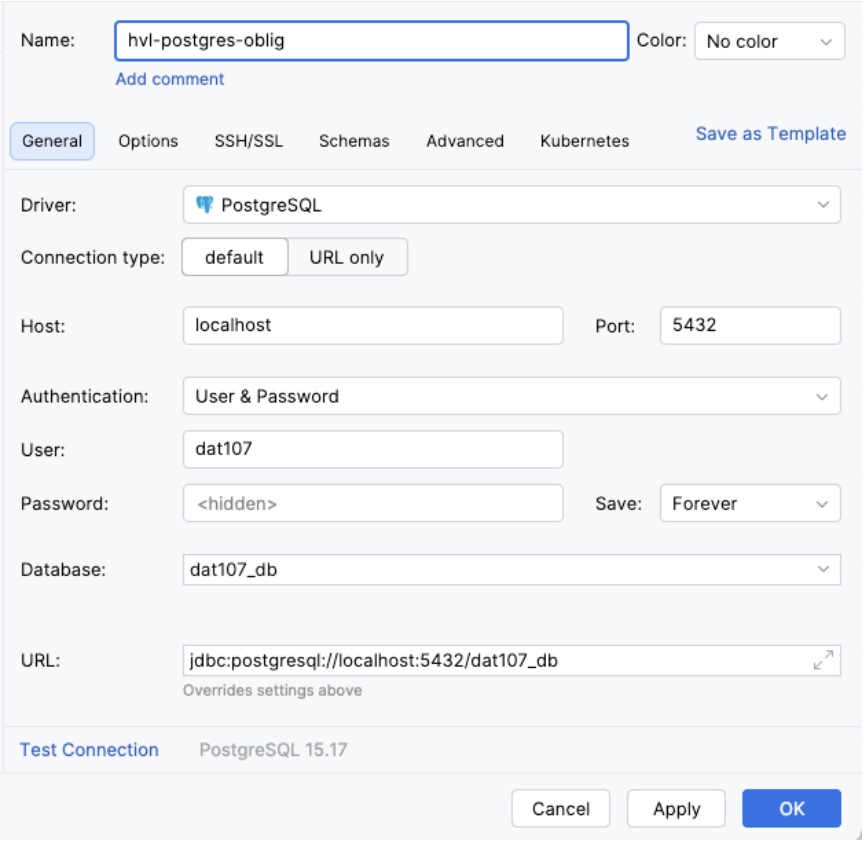
For å starte både PostgreSQL og MongoDB databasen kan du/dere kjøre følgende:

```
#> cd obligatorisk
#> docker-compose up -d
```

Forberedelser: DataGrip

Opprett 2 nye data kilder (Data Source) i DataGrip – en for PostgreSQL og en for MongoDB databasen.

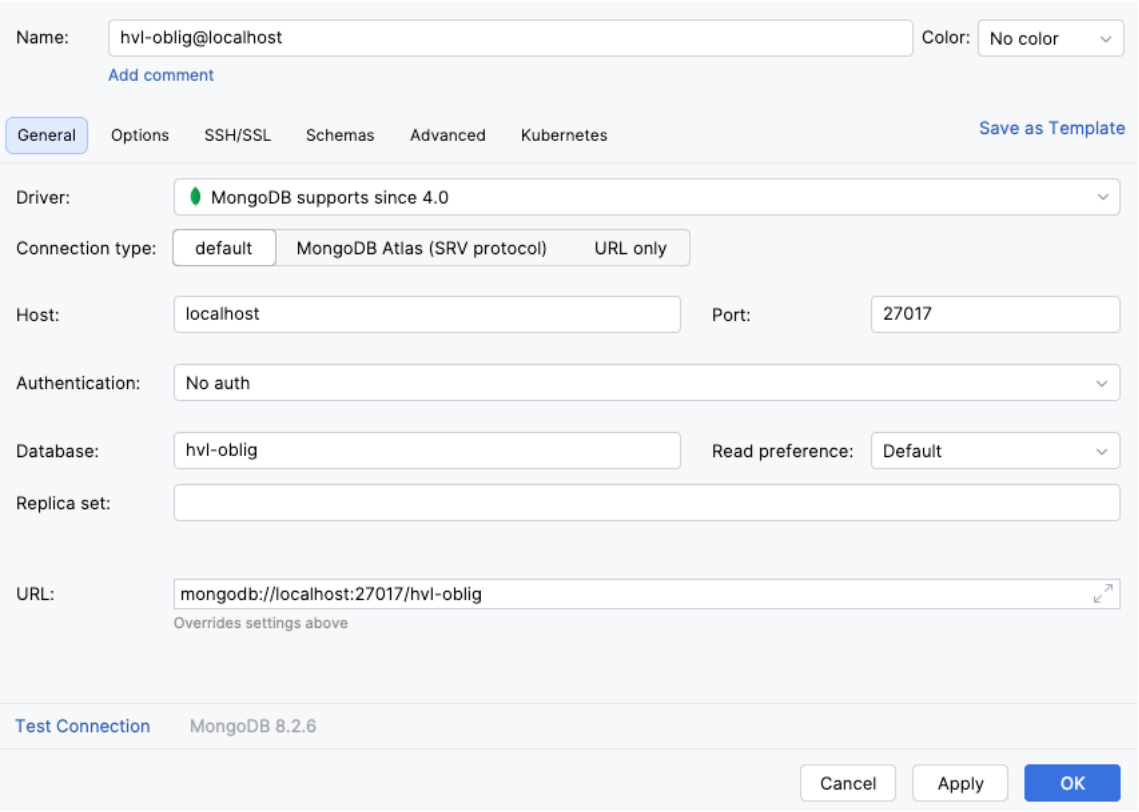
Data Source: PostgreSQL



Passord: dat107

(Oppgaven fortsetter på neste side.)

Data Source: MongoDB



(Oppgaven fortsetter på neste side.)

Oppgave 4.1 – XML, XML Skjema og XSLT

I denne oppgaven skal du/dere jobbe med XML, XML Skjema og XSL Transformasjoner.

a) Lag et XML-dokument (`lokallag.xml`) som representerer et lokallag med medlemmer og medlemsavgift-informasjon. Dokumentet skal inneholde:

- Informasjon om lokallaget (lagnavn, møtelokale med adresse)
- Informasjon om leder
- Minst 3 medlemmer med:
  - (Attributt) Medlemsnummer (navn på attributt "mId")
  - (Elementer) Navn (fornavn og etternavn)
  - (Elementer) Kontaktinformasjon (telefon og/eller epost)
  - (Elementer) Adresse (gateadresse, postnummer, poststed)
  - (Elementer) Status (aktiv/inaktiv)
- Medlemsavgift for de siste 3 årene for hvert medlem (år, betalt ja/nei, eventuell betalingsdato hvis avgiften er betalt)

b) Lag et XML Skjema (`lokallag.xsd`) som validerer XML-dokumentet fra oppgave a).
Koble XML-dokumentet fra oppgave a) til XML skjemaet (Tips! Bruk *noNamespaceSchemaLocation*).
XML Skjema skal sikre:

- Riktig struktur og rekkefølge på elementer
- At postnummer består av 4 siffer
- At år er et firesifret tall mellom 1900 og 2100
- (Valgfri oppgave) At medlemsnummer er unike

c) Lag en spørring i PostgreSQL som generer et XML dokument ut fra følgende spesifikasjon:

- XML dokumentet skal ha root-elementet <aktive-medlemmer>
- Spørringen skal hente ut alle aktive medlemmer i et gitt lokallag, sammen med status for medlemsavgift for gjeldende år. Lag et element <avgift-«gjeldende år»/> med attributtene «betalt» (J/N) og «betalingsdato». Sist nevnte skal kun vises hvis avgiften er betalt.
- Resultatet skal inkludere medlemsnummer («mId» attributt), fornavn, etternavn, og status for årets avgift.

Tips! Bygg opp SQL-en steg for steg. Test spørringen for hver gang du/dere legger til noe nytt. Bruk EXTRACT(YEAR from CURRENT_DATE) for å finne gjeldende år.

(Oppgaven fortsetter på neste side.)

Oppgave 4.2 – JSON

I denne oppgaven skal vi jobbe med JSON.

a) I denne oppgaven skal du/dere lage et JSON-dokument (`lokallag.json`) som representerer samme informasjon som i XML-dokumentet fra oppgave 4.1a, men i JSON-format.

Strukturen skal være logisk og lett å navigere, og bør inkludere:

- Informasjon om lokallaget
- Informasjon om leder
- Minst 3 medlemmer med:
  - Medlemsnummer
  - Navn (fornavn og etternavn)
  - Kontaktinformasjon (telefon og/eller epost)
  - Adresse (gateadresse, postnummer, poststed)
  - Status (aktiv/inaktiv)
- Medlemsavgift for de siste 3 årene for hvert medlem (år, betalt ja/nei, eventuell betalingsdato hvis avgiften er betalt)

b) I denne oppgaven skal du/dere skrive en spørring i PostgreSQL som returnerer et JSON-dokument (*NB! Du/dere trenger kun å besvare oppgaven med selve SQL-setningen).*

Spørringen skal hente ut alle aktive medlemmer i lokallag 1 (med medlemsnummer, fornavn, etternavn), sammen med status for medlemsavgift for gjeldende år («betalt» (J/N) og «betalingsdato»). Betalingsdato skal kun vises hvis avgiften er betalt.

Tips! Bygg opp SQL-en steg for steg. Test spørringen for hver gang du/dere legger til noe nytt. Bruk EXTRACT(YEAR from CURRENT_DATE) for å finne gjeldende år.

(Oppgaven fortsetter på neste side.)

Oppgave 4.3 - NoSQL

Generelt

Foreningen ønsker å lage en web-løsning for å håndtere påmeldinger til arrangementer. De ønsker å bruke en NoSQL database for dette formålet. I tillegg til informasjon om arrangementer og påmeldinger, skal data fra den gamle løsningen (data fra tabellene: lokallag, medlem og medlemsavgift) importeres inn i den nye løsningen.

a) Hvilken NoSQL database (alternativer: Neo4j, Cassandra, Redis, MongoDB) ville du valgt?
Gi en **kort** begrunnelse for valget ditt (dvs. hvorfor du mener at dette er en egnet database for formålet, og hvorfor de andre NoSQL alternativene ikke er like godt egnet).

MongoDB

Det blir etter hvert besluttet at MongoDB skal brukes som NoSQL database for denne løsningen.

b) I denne oppgaven skal du/dere utvide MongoDB databasen (hvl-oblig) som du/dere opprettet under forberedelsene (Se eget avsnitt «Forberedelser»).

Lag en passende datamodell for å lagre informasjon om arrangementer og påmeldinger i MongoDB databasen. Du/dere velger selv hvilke felter (nøkler og verdier) som skal lagres.

Forklar hvorfor du eventuelt velger «Embedding» og/eller «Linking».

c) Legg til eksempel data i MongoDB databasen som bruker designet fra forrige oppgave.
Du skal legge til 2 arrangementer med 3 påmeldinger til hvert arrangement.

Oppgaven besvares med:
- MongoDB kommandoene du/dere benytter for å legge inn data i databasen
- Screenshots av *find()* kommandoene som viser at dataene er lagt inn i MongoDB databasen.

(Oppgaven fortsetter på neste side.)

Oppgave 4.4 - MongoDB Query API

Under skal du skrive spørringer med MongoDB Query API-et for følgende:

NB! Du/dere trenger kun å svare med selve MongoDB Query uttrykket, f.eks. «db.lokallag.find()». Du/dere trenger ikke å lime in screenshots av resultatene.

a) Legge til et nytt medlem i lokallag 1 (_id=1).

b) Legge til 2 nye lokallag (_id = 2 og 3).

c) Slette lokallag 3 (_id=3).

d) Finn alle lokallagene i databasen.

e) Finn alle medlemmer i lokallag 1 (_id=1).

f) Finn de 3 medlemmene med lavest medlemsnummer i lokallag 1 (_id=1), og sorter stigende på _id.

g) Finn alle medlemmer som bor på et bestemt poststed (f.eks. “Bergen”).

h) Finn alle medlemmer som er aktive og har betalt medlemsavgift for 2025.

i) Finn alle medlemmer som har fornavn som starter på “A” eller etternavn som starter på “J”.

j) Finn antall medlemmer i lokallag 1 (_id=1).

k) (valgfri oppgave) Finn antall medlemmer for hvert lokallag.

l) (valgfri oppgave) Legge til betalingsinformasjon for 2027 for alle medlemmer i lokallag 1. Informasjonen skal merkes med «ikke betalt» (dvs. betalt = ‘N’).

God jobbet!